Gebze Technical University

CSE344 SYSTEM PROGRAMMING

-

HW4 REPORT

Veysel Cemaloğlu

200104004107

Problem Overview

The task at hand is to create a multithreaded C program that efficiently copies files from a source directory to a destination directory. This program must handle various types of files (regular files, directories, and FIFOs) and must do so in a way that maximizes throughput while ensuring thread safety. Additionally, the program must be robust enough to handle signals (such as SIGINT and SIGTSTP) and terminate gracefully. This involves the use of mutexes and condition variables to manage shared resources and synchronization between threads.

Compiling and Running

Compile the code:

```
make
```

Run the program:

```
./MWCp <buffer_size> <num_customers> <source_dir> <destination_dir>
```

Clean the object files:

```
make clean
```

PROBLEM SOLVING APPROACH

1. Checking the Given Paths

Before initiating any file operations, the program first verifies the validity of the source and destination paths. This is crucial to avoid potential issues such as attempting to copy files into a non-existent directory or copying a directory into itself.

- **Absolute Path Resolution**: The `realpath` function is used to resolve the absolute paths of the source and destination directories. This ensures that the program operates on the correct directories regardless of the relative paths provided by the user.
- **Parent Directory Check**: The program checks if the destination directory exists. If it does not, the program attempts to create the necessary parent directories.
- **Self-Copy Prevention**: The program checks if the destination directory is a subdirectory of the source directory. Copying a directory into itself would result in infinite recursion, so this situation is detected and handled appropriately.

2. Declaration of the Buffer

A circular buffer (queue) is used to store file descriptors that need to be processed (copied). This buffer serves as the communication channel between the producer (manager) thread (which enqueues file descriptors) and consumer (worker) threads (which dequeue file descriptors and perform the actual file copying).

- **Buffer Structure**: The buffer is implemented using a structure that maintains an array of file descriptors, along with indices to keep track of the front and rear of the queue. This allows for efficient enqueue and dequeue operations.
- **Buffer Capacity**: The buffer's capacity is specified by the user and dictates how many file descriptors can be stored in the buffer at any given time.

3. Creating Threads

The program uses one producer (manager) thread and multiple consumer (worker) threads to achieve parallelism.

- **Producer (Manager) Thread**: Responsible for traversing the source directory, opening files, and enqueuing file descriptors into the buffer.
- **Consumer (Worker) Threads**: Responsible for dequeuing file descriptors from the buffer and performing the actual file copy operations.

4. Mutexes and Condition Variables

To ensure thread safety and synchronization, mutexes and condition variables are used extensively.

- **Mutexes**: Protect shared resources such as the buffer, standard output, and file descriptor counters.
- **Condition Variables**: Facilitate communication between threads, allowing them to wait for certain conditions to be met (e.g., buffer not being full for the producer, buffer not being empty for the consumers).

5. Producer (Manager) Thread

The producer thread performs the following tasks:

1. **Directory Traversal**: Recursively traverses the source directory.
2. **File Descriptor Management**: Opens source and destination files, then enqueues their file descriptors into the buffer.
3. **Directory Creation**: Creates corresponding directories in the destination path.
4. **FIFO Handling**: Handles FIFO files by creating them in the destination directory.

6. Consumer (Worker) Threads

The consumer threads perform the following tasks:

1. **Buffer Dequeue**: Dequeues file descriptors from the buffer.
2. **File Copy**: Reads from the source file descriptor and writes to the destination file descriptor in chunks, ensuring that all data is copied correctly.
3. **Resource Management**: Closes file descriptors and frees allocated memory once the copy operation is complete.

7. Handle Signals

The program includes signal handlers to manage termination signals (SIGINT and SIGTSTP).

- **Signal Handling**: When a termination signal is received, the signal handler sets a flag to indicate that the program should terminate. This flag is checked by both producer and consumer threads to ensure they exit gracefully.
- **Broadcasting Condition Variables**: The signal handler broadcasts condition variables to wake up any waiting threads, allowing them to exit.

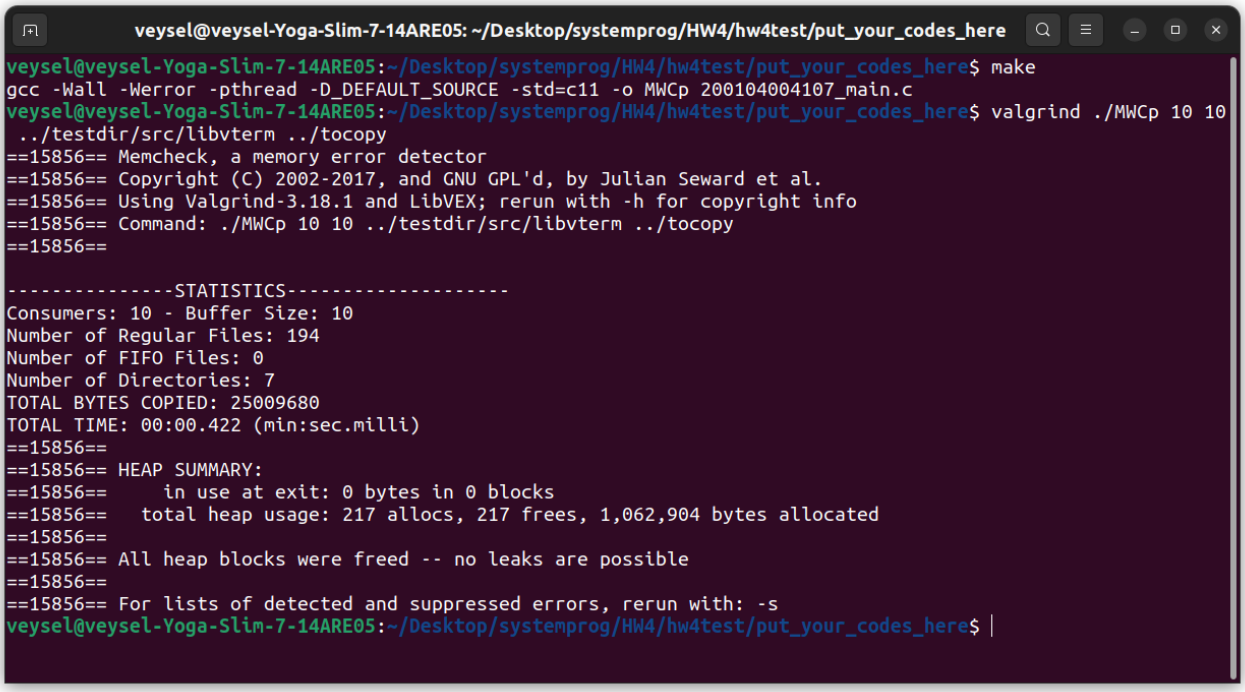
8. Terminating Threads

Graceful termination of threads is essential to ensure that all resources are properly released and no data is lost.

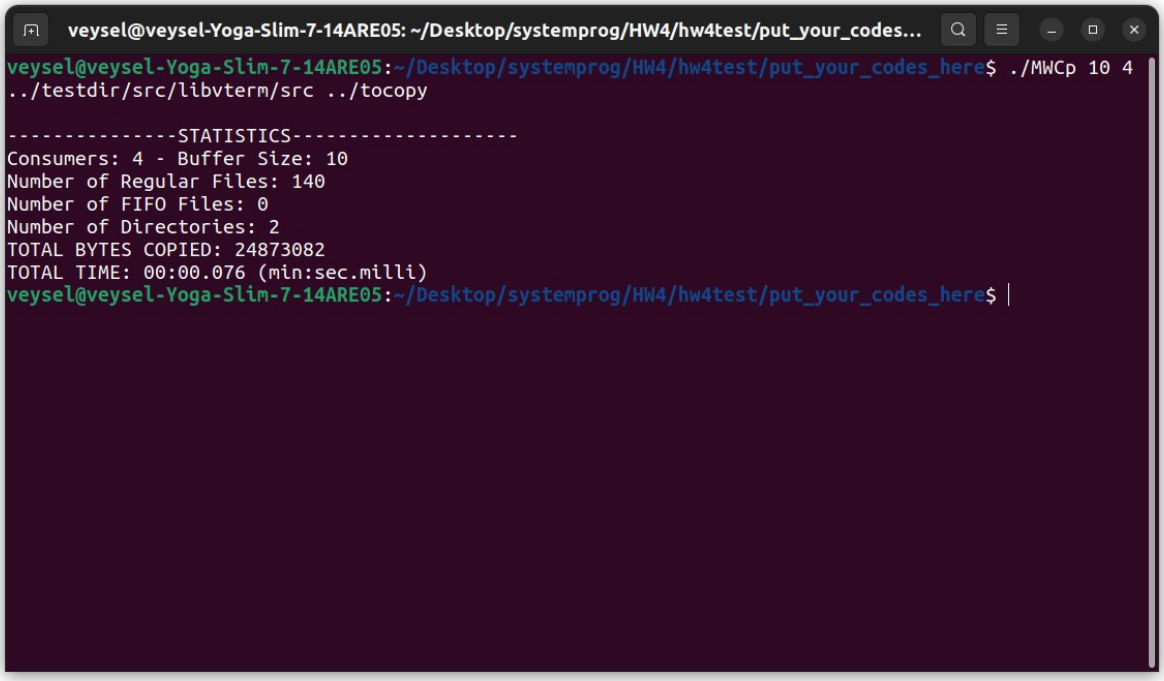
- **Thread Joining**: The main function waits for all threads to complete their execution using `pthread_join`.
- **Resource Cleanup**: Mutexes and condition variables are destroyed, and dynamically allocated memory is freed.
- **Statistics Printing**: Upon completion, the program prints statistics such as the number of files copied, the total bytes copied, and the total time taken for the operation.

TEST CASES:

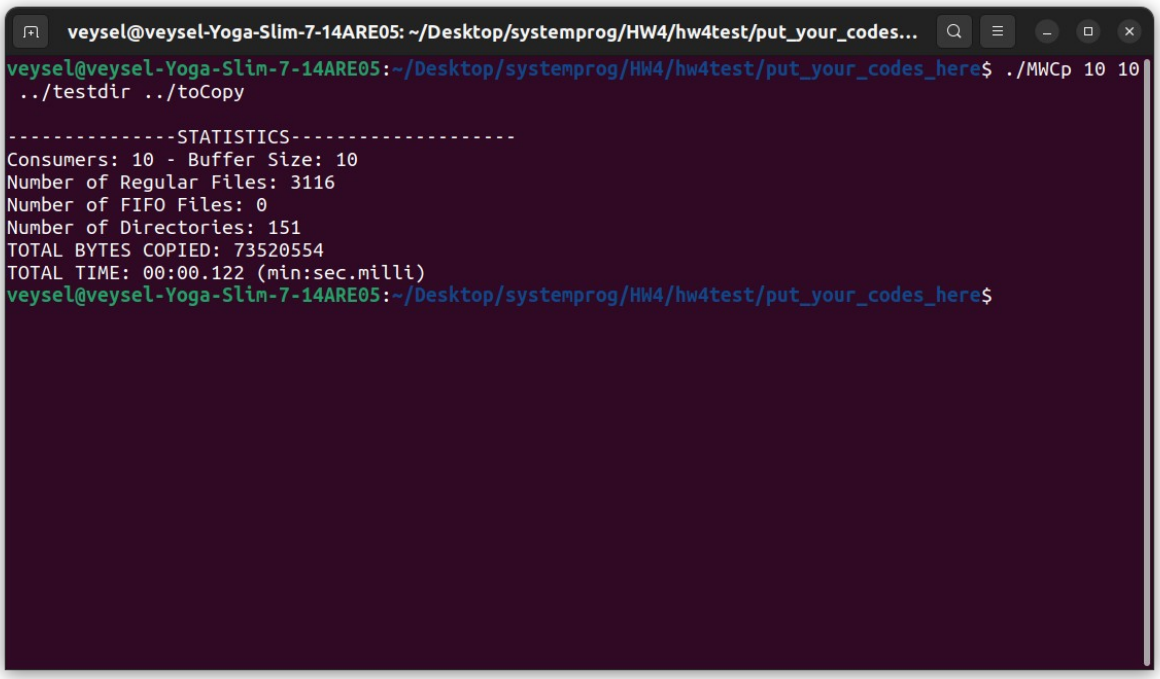
Test1: valgrind ./MWCp 10 10 ../testdir/src/libvterm ../tocopy :



Test2: ./MWCp 10 4 ../testdir/src/libvterm/src ../tocopy :



Test3: ./MWCp 10 10 ../testdir ../toCopy :



CONCLUSION

By using a combination of multithreading, synchronization primitives, and signal handling, the program efficiently copies files from a source directory to a destination directory. The use of a buffer for file descriptors allows for smooth communication between producer (manager) and consumer (worker) threads, while mutexes and condition variables ensure thread safety and coordination. This approach leverages the strengths of multithreading to achieve high throughput and robustness in file copy operations.